# CSE443 Project Deliverable #1
# Requirements Analysis Document

**Due Date: Oct 30th, 2026, 23:59**

**Purpose**: This assignment aims to define what the system will do and to analyze how the system will be structured conceptually using object-oriented modeling techniques. You will produce a Requirements Analysis Document supported by UML diagrams that describe system functionality and structure at the analysis level.

**Deliverables**: Each team must submit a comprehensive document as given in the template. A brief description of the template is as follows:

1. **Introduction**

This section will provide a brief overview of the function of the system and the reasons for its development, its scope, and references to the development context (e.g., reference to the problem statement written by the client, references to existing systems, feasibility studies). The introduction also includes the objectives and success criteria of the project.

2. **Current system**

This section describes the current state of affairs. If the new system will replace an existing system, this section describes the functionality and the problems of the current system. Otherwise, this section describes how the tasks supported by the new system are accomplished now.

3. **Proposed System**

The third section documents the functional and non-functional requirements elicitation and the analysis model of the new system.

Organized by modules or use cases, requirements follow the following rules:

- Numbered list of functional requirements (e.g., *FR-1, FR-2, ...*)
- Each written in a **EARS requirement sentence structure** (e.g., *"The system shall…"*)

This section will also include system models, including use case models, scenarios, object model, dynamic models and user interface mock-ups.

4. **Glossary of Terms**

5. **Priority**

6. **Traceability**

**Delivery Method**: Deliver your work in a Github repository. Any late submissions will be graded accordingly.

**Other Notes**:

- Use consistent naming between requirements, use cases, and classes.
- Give unique requirement IDs for each requirement.
- Follow requirements specification writing rules as we discussed in the lecture.
- Keep the domain model conceptual — no methods or database elements yet.
- Ensure traceability: each use case should link to one or more functional requirements.
- Use UML-compliant notation and clear labeling for all diagrams.